

Week 12

Task 1: To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

CODE:

```
saichaitanya@GeethzzSai: ~/s × + v
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define JOBS 4

int main()
{
    pid_t pids[JOBS];
    int status;
    time_t start, end;

    printf("Starting %d pipeline jobs...\n", JOBS);

    start = time(NULL);

    for (int i = 0; i < JOBS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0)
        {
            printf("Job %d started. PID = %d\n", i + 1, getpid());

            sleep(i + 1);

            printf("Job %d completed. PID = %d\n", i + 1, getpid());

            return 0;
        }
    }
}
```

```

        return 0;
    }
}

for (int i = 0; i < JOBS; i++)
{
    waitpid(pids[i], &status, 0);

    if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
    {
        printf("Job %d: SUCCESS\n", i + 1);
    }
    else
    {
        printf("Job %d: FAILED\n", i + 1);
    }
}

end = time(NULL);

printf("\nAll jobs completed.\n");

printf("Total execution time: %ld seconds\n",
        end - start);

printf("Pipeline stability check: PASSED\n");

return 0;
}
|

```

OUTPUT:

```

saichaitanya@GeethzzSai: ~/s  ×  +  ∨
saichaitanya@GeethzzSai:~$ mkdir skill_week12
saichaitanya@GeethzzSai:~$ cd skill_week12
saichaitanya@GeethzzSai:~/skill_week12$ nano 1.c
saichaitanya@GeethzzSai:~/skill_week12$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/skill_week12$ ./1
Starting 4 pipeline jobs...
Job 1 started. PID = 598
Job 2 started. PID = 599
Job 3 started. PID = 600
Job 4 started. PID = 601
Job 1 completed. PID = 598
Job 1: SUCCESS
Job 2 completed. PID = 599
Job 2: SUCCESS
Job 3 completed. PID = 600
Job 3: SUCCESS
Job 4 completed. PID = 601
Job 4: SUCCESS

All jobs completed.
Total execution time: 4 seconds
Pipeline stability check: PASSED
saichaitanya@GeethzzSai:~/skill_week12$ |

```

Task 2: To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

.CODE:

```
saichaitanya@GeethzzSai: ~/s × + v
GNU nano 8.7.1 2.c *
# Large Pipeline Execution System

## 1. Architecture

The system contains:

User
|
v
Main Process
|
+-----> Job 1
|
+-----> Job 2
|
+-----> Job 3
|
+-----> Job 4
|
v
Wait and Monitor
|
v
Performance Report

## 2. Execution Flow

1. Main process starts.
2. Number of jobs is defined.
3. fork() creates child processes.
4. Each child performs its assigned work.
5. Parent monitors every child using waitpid().
6. Exit status is checked.
7. Execution time is calculated.
8. Final stability result is displayed.
```

3. Important System Calls

fork()

- Creates a **new** child process.

waitpid()

- Waits **for** a specific child process.

getpid()

- Returns the process **ID**.

sleep()

- Simulates work performed by a job.

4. Execution Path

Start

|

v

Create Jobs

|

v

Run Jobs

|

v

Monitor Processes

|

v

Check Exit Status

|

+---- Success ----> Record Result

```
saichaitanya@GeethzzSai: ~/s  ×  +  v
GNU nano 8.7.1                2.c *
Measure Time
|
v
Display Final Report
|
v
End

## 5. Design Decisions

1. Multiple child processes are used to simulate multiple jobs.
2. waitpid() is used to monitor individual processes.
3. Exit status is checked to detect failures.
4. time() is used to measure execution duration.
5. The implementation is kept simple for testing process management.

## 6. Testing

The program is compiled using GCC and executed from the terminal.

Expected results:

- All jobs should start.
- All jobs should complete.
- Successful jobs should be reported.
- Total execution time should be displayed.
- Stability check should be displayed.

## 7. Conclusion

The program demonstrates process creation, process monitoring,
failure detection, execution-time measurement, and basic
pipeline/job stability analysis.
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week12$ nano 2.c
saichaitanya@GeethzzSai:~/skill_week12$ cat 2.c
# Large Pipeline Execution System

## 1. Architecture

The system contains:

User
|
v
Main Process
|
+-----> Job 1
|
+-----> Job 2
|
+-----> Job 3
|
+-----> Job 4
|
v
Wait and Monitor
|
v
Performance Report

## 2. Execution Flow

1. Main process starts.
2. Number of jobs is defined.
3. fork() creates child processes.
4. Each child performs its assigned work.
5. Parent monitors every child using waitpid().
6. Exit status is checked.
```

Task 3: To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

CODE:

```
saichaitanya@GeethzzSai: ~/s × + v
GNU nano 8.7.1 3.c *
CC = gcc
CFLAGS = -Wall
TARGET = bin/large_pipeline
SOURCE = src/large_pipeline.c

all:
    $(CC) $(CFLAGS) $(SOURCE) -o $(TARGET)

run: all
    ./${TARGET}

clean:
    rm -f $(TARGET)

check:
    @echo "Checking project structure..."
    @test -f src/large_pipeline.c && echo "Source file: OK" || echo "Source file: MISSING"
    @test -f architecture.md && echo "Documentation: OK" || echo "Documentation: MISSING"
    @test -d src && echo "src/: OK" || echo "src/: MISSING"
    @test -d bin && echo "bin/: OK" || echo "bin/: MISSING"
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week12$ nano Makefile
saichaitanya@GeethzzSai:~/skill_week12$ make
gcc -Wall 1.c -o 1
saichaitanya@GeethzzSai:~/skill_week12$ make run
gcc -Wall 1.c -o 1
./1
Starting 4 pipeline jobs...
Job 1 started. PID = 874
Job 2 started. PID = 875
Job 3 started. PID = 876
Job 4 started. PID = 877
Job 1 completed. PID = 874
Job 1: SUCCESS
Job 2 completed. PID = 875
Job 2: SUCCESS
Job 3 completed. PID = 876
Job 3: SUCCESS
Job 4 completed. PID = 877
Job 4: SUCCESS

All jobs completed.
Total execution time: 4 seconds
Pipeline stability check: PASSED
saichaitanya@GeethzzSai:~/skill_week12$ |
```

Task 4: To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

CODE:

```
GNU nano 8.7.1 4.c *
# Clean Build

make clean

## System Calls Used

- fork()
- waitpid()
- getpid()
- sleep()

## Testing

The project creates multiple child processes, monitors their
execution, checks their exit status, and measures execution time.

|
## Conclusion

The project demonstrates process creation, monitoring,
performance measurement, documentation, build automation,
and final project validation.
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week12$ nano 4.c
saichaitanya@GeethzzSai:~/skill_week12$ cat 4.c
# Clean Build

make clean

## System Calls Used

- fork()
- waitpid()
- getpid()
- sleep()

## Testing

The project creates multiple child processes, monitors their
execution, checks their exit status, and measures execution time.

## Conclusion

The project demonstrates process creation, monitoring,
performance measurement, documentation, build automation,
and final project validation.
saichaitanya@GeethzzSai:~/skill_week12$ |
```